Erste Schritte in Python

Parameter	Kursinformationen
Veranstaltung:	`Erhebung, Analyse und Visualisierung digitaler Daten`
Semester	`Wintersemester 2026/27`
Hochschule:	`Technische Universität Freiberg`
Inhalte:	`Uhrzeiten in der Tabellenkalkulation, Von der Frage zum Pseudocode, Interpreter, Ausgabe, Rechnen, Variablen, Datentypen, Typumwandlung, Fehlermeldungen`
Link auf Repository:	https://github.com/TUBAF-IfI-LiaScript/VL_EAVD/blob/master/02_ErsteSchritte.md
Autoren	Sebastian Zug & Bernhard Jung & Claude Code (Opus 5.5)

Leitfrage: *Was macht der Rechner mit „-1,9“ aus unserer Datei?*

Fragen an die heutige Veranstaltung …

- Wie wird aus einem Programmtext etwas, das der Rechner ausführt?
- Wie speichert ein Programm Werte, und wie rechnet es damit?
- Warum ist `"6,26"` etwas anderes als `6.26`?
- Was sagt mir eine Fehlermeldung — und wie gehe ich damit um?

> **Zum Nacharbeiten: die Ansicht „Lehrbuch“.** Im Hörsaal zeigen die Folien nur die Kernbotschaften. Die Erklärungen dazu stehen im Lehrbuch: oben rechts auf das Symbol mit den Schiebereglern klicken und „Lehrbuch“ wählen.

🗺️ Diese Vorlesung in der Kurslandkarte →

Heute schreiben Sie Ihre ersten Zeilen Python. Wir beginnen mit der Tabellenkalkulation und dem Pseudocode, damit der Übergang kein Sprung wird. Am Ende nehmen wir eine echte Zeile aus unserer Datei auseinander. Sie werden dabei Fehlermeldungen sehen, manche mit Absicht. Das ist kein Zeichen dafür,

dass etwas schiefläuft, sondern Teil der Arbeit.

Rückblick 1: Pseudocode Grundkonzepte

Ihre Hausaufgabe: *Den wärmsten Tag in* `fichtelberg_2024.csv` *finden, ohne* `MAX` *zu benutzen.* Zur Erinnerung: Die Datei hat 366 Zeilen, eine pro Tag.

fichtelberg_2024.csv

```
Datum;Tagesmittel;Minimum;Maximum
2024-01-01;-1,2;-1,9;-0,7
2024-01-02;-0,5;-2,4;2,6
2024-01-03;1,6;0,1;2,5
...
2024-08-13;21,3;16,9;26,9
...
```

Eine mögliche Lösung:

```
Merke dir den ersten Tag aus der Spalte "Maximum" als "bisher wärmster".
Für jeden weiteren Tag:
    Wenn sein Maximum höher ist als das des bisher wärmsten:
        Merke dir diesen Tag als "bisher wärmster".
Gib den bisher wärmsten Tag aus.
```

Das Ergebnis: Am 13. August 2024 war es mit 26,9 °C am wärmsten.

> Darin stecken schon fast alle Zutaten eines Programms: Man muss sich etwas **merken** (heute), etwas **vergleichen** und etwas **wiederholen** (nächste Woche). Heute kümmern wir uns um das Merken und Rechnen.

Ihre Formulierung darf anders aussehen, wichtig ist der Ablauf. `MAX` macht es im Grunde genauso: Es geht alle Werte durch und merkt sich den bisher größten. Sie haben also von Hand aufgeschrieben, was die Funktion Ihnen sonst abnimmt.

Rückblick 2: Unterschiedliche Datentypen

In Aufgabe 2 war der Zeitstempel die größte Hürde. Für die Tabellenkalkulation ist eine Uhrzeit eine **Zahl**, nämlich der Bruchteil eines Tages: 12:00 Uhr ist 0,5. Erst als Zahl lässt sie sich vergleichen.

Schritt	Formel (in A2 steht der Text `2026-10-19T16:15:27`)	Ergebnis
1. Uhrzeit abspalten	`=TEIL(A2;12;8)` in C2	`16:15:27`, aber noch Text
2. in eine Uhrzeit umwandeln	`=ZEITWERT(C2)` in D2	`0,6774`, als Uhrzeit formatiert `16:15:27`
3. vergleichen	`=D2>=ZEIT(16;15;0)`	`WAHR`

Alle Durchgänge `rein` ab 16:15 auf einmal:
`=ZÄHLENWENNS(B2:B32;"rein";D2:D32;">="&ZEIT(16;15;0))` ergibt **8**.

> Hat LibreOffice den Zeitstempel beim Öffnen schon als Datum mit Uhrzeit erkannt (rechtsbündig), entfallen Schritt 1 und 2. Die Uhrzeit allein liefert dann `=REST(A2;1)`.

Intern zählt die Tabellenkalkulation die Tage seit Ende 1899. Der 19. Oktober 2026 ist Tag 46.314, und 16:15:27 Uhr ist der Bruchteil 0,6774 dieses Tages; zusammen ergibt das 46.314,6774. REST(A2;1) teilt durch 1 und behält nur den Rest, also den Tagesanteil. TEIL(A2;12;8) zählt dagegen Zeichen: ab dem zwölften Zeichen acht Stück. Das funktioniert nur, solange alle Zeitstempel genau gleich aufgebaut sind.

Von der Frage zum Pseudocode

Jede Auswertung beginnt mit einer Frage an einen Datensatz. Vier Fragen an sich selbst führen von dort zum Pseudocode:

Fragen Sie sich …	im Beispiel
1. Was steht in **einer** Zeile?	eine Person: Nachname, Vorname, Studiengang
2. Was muss ich mir **merken**?	eine Anzahl, zu Beginn 0
3. Was tue ich mit **jeder** Zeile?	Beginnt der Nachname mit „A“, die Anzahl erhöhen
4. Was gebe ich am **Ende** aus?	die Anzahl

Beispiel: In der Klausur sitzt jeder Anfangsbuchstabe in einem eigenen Block. Wie viele Plätze müssen wir für Block A freihalten? Das heißt: *Wie viele Personen auf der Teilnehmerliste haben einen Nachnamen, der mit A beginnt?*

```
Setze anzahl auf 0.
Für jede Zeile der Teilnehmerliste:
    Wenn der Nachname mit "A" beginnt:
        Erhöhe anzahl um 1.
Gib anzahl aus.
```

> Frage 2 ist die wichtigste: Was man sich merken muss, wird heute zur **Variablen**.

Die vier Fragen sind ein Rezept für den Anfang, kein Gesetz. Frage 1 klingt banal, entscheidet aber, worüber die Wiederholung läuft: über Personen, Tage, Messpunkte oder Buchungen. Frage 3 enthält fast immer eine Bedingung, und die muss man ganz genau formulieren. Wie genau, zeigt die nächste Folie.

Was heißt „beginnt mit A“?

Auf der Teilnehmerliste stehen unter anderem diese vier Nachnamen. Legen Sie sich fest, **bevor** Sie die Auflösung ansehen: Welche zählt der Algorithmus, wenn er **genau** das tut, was dasteht?

- [ ] Ahrens
- [ ] Åberg
- [ ] von Arnim
- [ ] auf der Heide

> Eindeutig heißt nicht: so gemeint. Der Algorithmus beantwortet genau die Frage, die in ihm steht.

Für die Klausur hätte das Folgen: Wer „von Arnim“ heißt und in Block A sucht, findet dort keinen freigehaltenen Platz. Dasselbe Problem kennen Sie aus Vorlesung 01: Der Algorithmus mit der verrutschten Zeile war eindeutig und ausführbar, er beantwortete nur eine andere Frage als gemeint. Bei echten Daten stecken solche Fälle überall, etwa Groß- und Kleinschreibung, Leerzeichen am Anfang oder Sonderzeichen. Deshalb lohnt es sich, vor dem Programmieren ein paar echte Zeilen anzusehen.

Üben: Fragen aus Ihrem Fach

⏱ **5 Minuten, zu zweit.** Wählen Sie eine Frage, am besten aus Ihrem Fach. Gehen Sie die vier Fragen durch und schreiben Sie Pseudocode.

Fach	Eine Zeile des Datensatzes	Frage
Umwelt	Datum; Feinstaub im Tagesmittel in µg/m³	Hielt die Messstation den Grenzwert ein? Über 50 µg/m³ ist an höchstens 35 Tagen im Jahr erlaubt.
Werkstoffe	Messpunkt eines Zugversuchs: Dehnung in %; Kraft in kN	Die Probe gilt als gerissen, sobald die Kraft nach dem Höchstwert unter die Hälfte davon fällt. Bei welcher Dehnung war das?
Wirtschaft	Datum; Filiale; Betrag in €	Welche Filiale machte im März den höchsten Umsatz?
Energie	Uhrzeit; Leistung in kW, alle 15 Minuten gemessen	Wie oft stieg die Leistung der Werkhalle von einer Messung zur nächsten um mehr als 20 kW?
Schon Erfahrung?	Uhrzeit; Ereignis `start` oder `stopp` einer Maschine	Wie lange lief die Maschine insgesamt?

Beim **Zugversuch** wird eine Materialprobe langsam auseinandergezogen, bis sie reißt. Die **Dehnung** sagt, um wie viel Prozent sie dabei länger geworden ist.

Jede Frage verlangt eine andere Idee: zählen und danach entscheiden, die Daten zweimal durchgehen, pro Gruppe zusammenfassen, die vorherige Zeile im Gedächtnis behalten und sich einen Zustand merken. Einige dieser Muster bekommen in Vorlesung 04 eigene Namen. Feinstaub sind winzige Staubteilchen in der Luft, die man einatmet. Für sie gilt in der EU tatsächlich: Das Tagesmittel darf höchstens an 35 Tagen im Jahr über 50 Mikrogramm pro Kubikmeter liegen.

Lösungsvorschläge

Es gibt nicht die eine richtige Lösung. Prüfen Sie Ihre, indem Sie sie mit drei, vier erfundenen Zeilen von Hand ausführen.

Umwelt: Grenzwert für Feinstaub ›

Werkstoffe: Riss im Zugversuch ›

Wirtschaft: beste Filiale im März ›

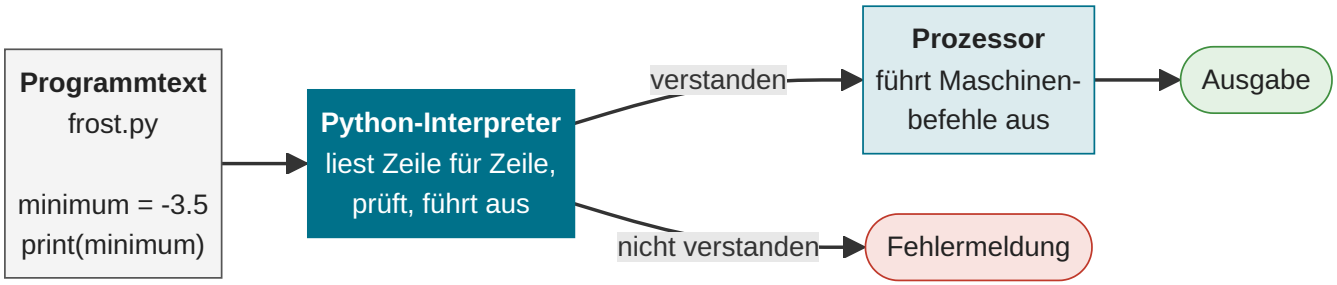
Energie: Sprünge in der Leistung ›

Schon Erfahrung? Laufzeit einer Maschine ›

Achten Sie beim Vergleichen weniger auf die Formulierung als auf zwei Dinge: Haben Sie sich das Richtige gemerkt, also eine Anzahl, eine Summe oder eine ganze Zeile? Und steht jeder Schritt an der richtigen Stelle, innerhalb oder außerhalb der Wiederholung? Genau daran scheiterte letzte Woche die verrutschte Zeile.

Vom Text zur Ausführung

Ein Prozessor versteht nur **Maschinenbefehle**, Pseudocode ist für ihn unverständlich. Zwischen Ihrem Programmtext und dem Prozessor steht deshalb ein *Übersetzer*, bei Python der **Interpreter**:



Zwei Dinge sollten Sie sich merken:

1. Der Interpreter arbeitet **von oben nach unten**, eine Anweisung nach der anderen.
2. Stößt er auf etwas, das er nicht versteht, **bricht er ab** und sagt Ihnen, wo und warum.

Maschinenbefehle sind sehr einfache Anweisungen wie „lade Wert“, „addiere“ oder „springe“, gespeichert als Bitmuster. Von Hand schreibt das heute kaum noch jemand. Der Interpreter ist selbst ein Programm. Auf Ihrem Rechner haben Sie ihn in der ersten Übung installiert, hier im Browser läuft eine eigene Fassung davon, sie heißt Pyodide.

Warum Python?

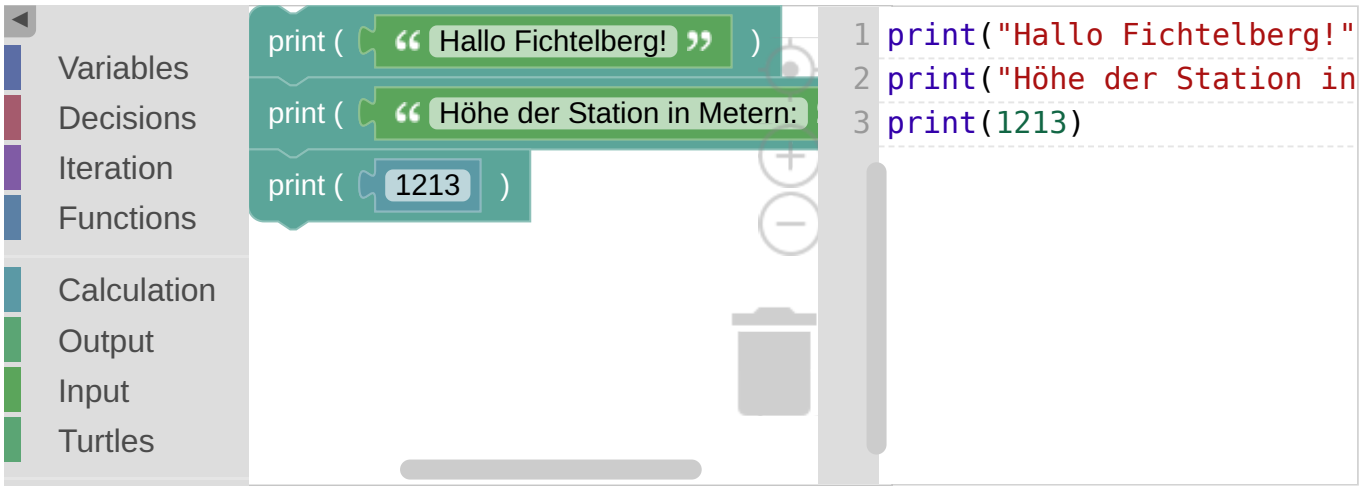
Python ist nicht die schnellste Sprache und nicht die einzige. Für unsere Zwecke — Daten einlesen, aufbereiten, auswerten, darstellen — ist sie aber die naheliegendste.

Python ist gut lesbar: Programme sehen fast aus wie der Pseudocode von letzter Woche. In der Wissenschaft ist es so verbreitet, dass es für fast jede Fachfrage eine fertige Bibliothek gibt. Es reicht vom Mikrocontroller in Vorlesung 08 bis zur Datenanalyse mit pandas, und es kostet nichts und läuft auf jedem Rechner.

Die erste Anweisung

Die Anweisung `print` gibt etwas aus. Was ausgegeben werden soll, steht in runden Klammern. Text steht in Anführungszeichen.

Nutzen Sie die Schaltflächen über dem Editor, um zwischen **Blöcken** und **Text** zu wechseln. Beide zeigen dasselbe Programm.



Ausprobieren

- Ergänzen Sie eine Zeile, die das Jahr des Messbeginns (1890) ausgibt.
- Was passiert, wenn Sie bei `"Hallo Fichtelberg!"` ein Anführungszeichen weglassen?
- `print` kann mehrere Dinge auf einmal ausgeben, getrennt durch Kommas: `print("Höhe:", 1213, "m")`

`print` bringt Python schon mit. Die Anführungszeichen sagen Python, dass Text folgt, den es nicht deuten soll; die Zahl `1213` steht deshalb ohne. Blöcke nutzen wir nur am Anfang: Sie verhindern Schreibfehler, sodass Sie sich auf den Ablauf konzentrieren können. Nach Vorlesung 07 arbeiten wir nur noch im Text.

Rechnen

Operator	Bedeutung	Beispiel	Ergebnis
`+`	Addition	`3 + 4`	`7`
`-`	Subtraktion	`3 - 4`	`-1`
`*`	Multiplikation	`3 * 4`	`12`
`/`	Division	`7 / 2`	`3.5`
`//`	ganzzahlige Division	`7 // 2`	`3`
`%`	Rest der Division	`7 % 2`	`1`
`**`	Potenz	`2 ** 10`	`1024`

Es gilt Punkt- vor Strichrechnung, Klammern gehen vor.

```
1  print(3 + 4 * 2)
2  print((3 + 4) * 2)
3  print(7 / 2)
4  print(7 // 2)
5  print(18900801 // 10000)
```

```
Another process is running, wait until finished
```

Die letzte Zeile ist kein Zufall: Das DWD-Datum `18900801` ganzzahlig durch `10000` geteilt ergibt das Jahr. Genau diesen Trick nutzt die Vier-Zeilen-Lösung aus der letzten Vorlesung.

> ⚠ Warning
>
> **Dezimalpunkt, nicht Dezimalkomma!** Python ist eine englischsprachige Sprache. `6.26` ist eine Zahl. `6,26` ist etwas völlig anderes:

```
1  print(6.26 + 1)
2  print(6,26 + 1)
```

```
Another process is running, wait until finished
```

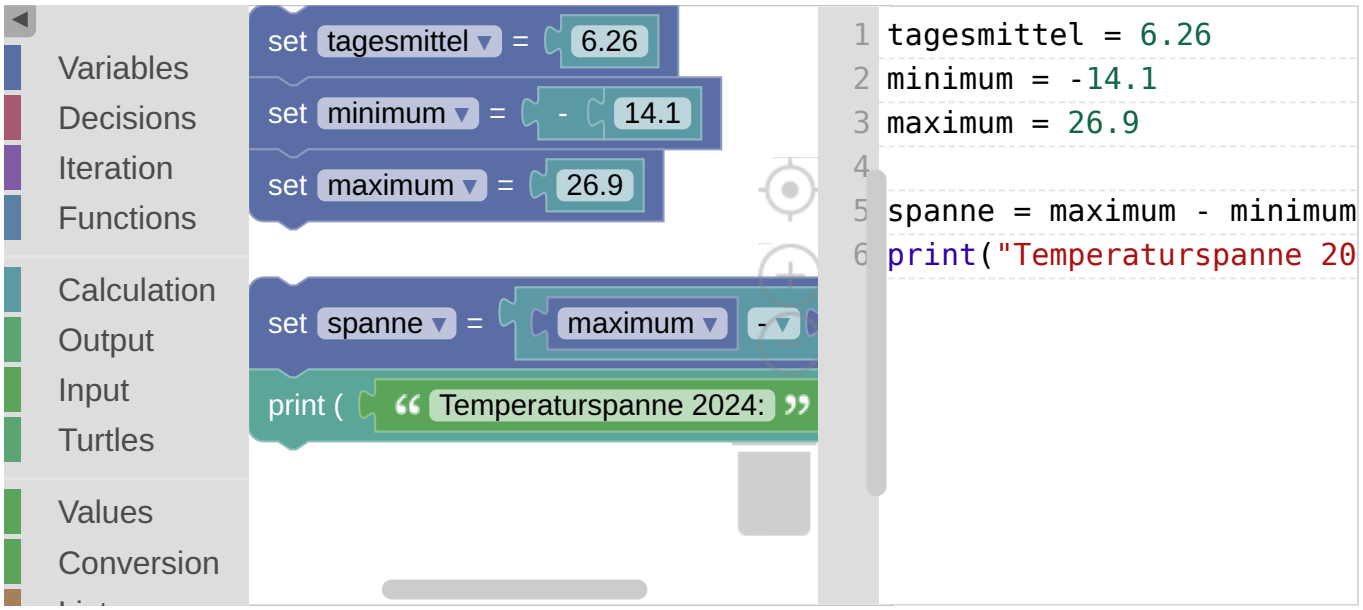
Die zweite Zeile gibt `6 27` aus. Python hat zwei Werte gesehen — `6` und `26 + 1` — und beide ausgegeben. **Keine Fehlermeldung, aber ein falsches Ergebnis.** Das ist die gefährlichste Art von Fehler.

`/` liefert immer eine Kommazahl, auch bei `8 / 2`. `%` liefert den Rest einer Division. Damit prüft man zum Beispiel, ob eine Zahl gerade ist: `n % 2` ergibt dann 0.

Python liest das Komma als Trennzeichen zwischen zwei Werten, genau wie in `print("Höhe:", 1213)`. Deutsche Zahlen in Dateien sind deshalb eine ständige Falle. Am Ende der Vorlesung begegnet sie uns wieder.

Variablen

Eine **Variable** ist ein Name, unter dem sich ein Programm einen Wert merkt.




```
Another process is running, wait until finished
```

Die Werte stammen aus dem Jahr 2024: Jahresmittel, tiefstes Minimum und höchstes Maximum. Eine Variable ähnelt einer benannten Zelle in der Tabellenkalkulation, mit einem wichtigen Unterschied, den Sie gleich sehen werden. Eine Variable entsteht in dem Moment, in dem ihr zum ersten Mal ein Wert zugewiesen wird.

Das Gleichheitszeichen ist keine Gleichung

In der Mathematik ist `x = x + 1` ein Widerspruch. In Python ist es eine ganz normale Anweisung:

```
1 zaehler = 0
2 zaehler = zaehler + 1
3 zaehler = zaehler + 1
4 print(zaehler)
```

```
2
```

Lesen Sie `=` als „wird zu“. Rechts wird zuerst ausgerechnet, dann wird das Ergebnis unter dem Namen links abgelegt. Der alte Wert ist danach weg.

Programme im Kopf ausführen

Die wichtigste Fähigkeit dieses Kurses — und eine typische Klausuraufgabe — ist, ein Programm Zeile für Zeile im Kopf nachzuvollziehen. Dazu legt man eine Tabelle an, eine Spalte pro Variable:

```
a = 5
b = a * 2
a = a + b
b = b - 1
print(a, b)
```

Was gibt das Programm aus? Legen Sie sich fest, **bevor** Sie die Lösung ansehen.

nach Zeile	`a`	`b`
1	5	–
2	5	10
3	15	10
4	15	9

Die Ausgabe lautet also `15 9`.

`zaehler = zaehler + 1` wird Ihnen ab nächster Woche ständig begegnen: So zählt ein Programm mit. Rechts steht der alte Wert, links landet der neue.

Das ist das Ablaufprotokoll aus Vorlesung 01, jetzt für Python. Eine Zeile pro Anweisung, eine Spalte pro Variable. Die Zeile mit `print` ändert keine Variable, deshalb fehlt sie in der Tabelle.

Typische Fehlvorstellung: Variablen rechnen nach

Typische Fehlvorstellung: *„Eine Variable merkt sich die Rechnung — wie eine Formel in der Tabellenkalkulation.“*

```
1  minimum = -3.5
2  maximum = 2.6
3  spanne = maximum - minimum
4  maximum = 8.0
5  print(spanne)
```

```
6.1
```

Legen Sie sich fest, **bevor** Sie das Programm ausführen: Was wird ausgegeben?

- ◯ `6.1`
- ◯ `11.5`
- ◯ eine Fehlermeldung, weil `maximum` zweimal belegt wird

Hier zeigt sich der Unterschied aus Vorlesung 01: Die Tabellenkalkulation ist eingabengetrieben, ein Skript läuft der Reihe nach. Python merkt sich das Ergebnis einer Rechnung, nicht ihren Weg.

Namen wählen

Ein Variablenname

- besteht aus Buchstaben, Ziffern und dem Unterstrich `_`,
- beginnt **nicht** mit einer Ziffer,
- unterscheidet Groß- und Kleinschreibung: `Minimum` und `minimum` sind zwei verschiedene Variablen,
- darf kein reserviertes Wort wie `if`, `for`, `while` oder `True` sein.

Name	erlaubt?	gut?
`tagesmittel`	ja	ja — sagt, was drin ist
`t`	ja	nur für ganz kurze Rechnungen
`max_temp_2024`	ja	ja — Wörter trennt man mit `_`
`2024_max`	**nein**	beginnt mit einer Ziffer
`höhe`	ja	besser `hoehe` — Umlaute machen früher oder später Ärger

Ein guter Name spart einen Kommentar. `x = 1213` erklärt nichts, `stationshoehe_m = 1213` alles.

Typische Fehlvorstellung: *„Der Rechner versteht, was ein Name bedeutet.“*

`frosttage = 0` zählt keine Frosttage. `mittelwert = 0` berechnet keinen Mittelwert. Namen sind nur Etiketten für Sie und andere Menschen. Was in einer Variable steht, bestimmen ausschließlich die Anweisungen, die Sie schreiben.

Python kennt rund 35 reservierte Wörter, einige davon lernen Sie nächste Woche kennen. Visual Studio Code färbt sie ein, Sie können sie also nicht übersehen. Umlaute sind erlaubt, machen aber Ärger, sobald Dateien zwischen Rechnern wandern.

Datentypen

Jeder Wert hat einen **Datentyp**. Der Typ entscheidet, was man mit dem Wert tun kann.

Typ	Bedeutung	Beispiel aus unserem Datensatz
`int`	ganze Zahl	`1358` (Stations-ID)
`float`	Kommazahl	`-14.1` (Minimum in °C)
`str`	Text (Zeichenkette)	`"Fichtelberg"`
`bool`	Wahrheitswert	`True` (war es ein Frosttag?)

Mit `type(...)` können Sie jeden Wert nach seinem Typ fragen:

```
1  print(type(1358))
2  print(type(-14.1))
3  print(type("Fichtelberg"))
4  print(type(-14.1 < 0))
5  print(type("18900801"))
```

```
<class 'int'>
<class 'float'>
<class 'str'>
<class 'bool'>
<class 'str'>
```

Der DWD schreibt die Stations-ID mit führender Null: `01358`. Als `int` geht die Null verloren, und ohne Anführungszeichen meldet Python sogar einen `SyntaxError`. Kennziffern wie Postleitzahlen sind deshalb Text, auch wenn sie aus Ziffern bestehen. `float` steht für Gleitkommazahl, `bool` ist nach dem Mathematiker George Boole benannt.

Text ist nicht Zahl

Wenn Sie eine Datei einlesen (Vorlesung 06), bekommen Sie immer **Text**. Auch wenn er aussieht wie eine Zahl:

```
1  minimum_aus_datei = "-14.1"
2  print(minimum_aus_datei + 1)
```

```
Traceback (most recent call last):
  File "/lib/python313.zip/_pyodide/_base.py", line 597, in
eval_code_async
    await CodeRunner(
    ...<9 lines>...
    .run_async(globals, locals)
  File "/lib/python313.zip/_pyodide/_base.py", line 411, in run_async
    coroutine = eval(self.code, globals, locals)
  File "<exec>", line 2, in <module>
TypeError: can only concatenate str (not "int") to str
```

Python weigert sich — zu Recht. Was soll `"-14.1" + 1` sein?

Die Lösung ist eine **Typumwandlung**:

Umwandlung	Beispiel	Ergebnis
`float(...)`	`float("-14.1")`	`-14.1`
`int(...)`	`int("1890")`	`1890`
`str(...)`	`str(1213)`	`"1213"`
`int(...)`	`int(6.9)`	`6` — schneidet ab, rundet nicht!

```
1  minimum_aus_datei = "-14.1"
2  minimum = float(minimum_aus_datei)
3  print(minimum + 1)
4
5  wert_aus_tabelle = "6,26"
6  print(float(wert_aus_tabelle))
```

```
-13.1
Traceback (most recent call last):
  File "/lib/python313.zip/_pyodide/_base.py", line 597, in
eval_code_async
    await CodeRunner(
    ...<9 lines>...
    .run_async(globals, locals)
  File "/lib/python313.zip/_pyodide/_base.py", line 411, in run_async
    coroutine = eval(self.code, globals, locals)
  File "<exec>", line 6, in <module>
ValueError: could not convert string to float: '6,26'
```

Die letzte Zeile scheitert. Ändern Sie sie so, dass sie funktioniert.

Tipp: Texte haben eine Methode `replace`: `"a,b".replace(",", ".")` ergibt `"a.b"`.

Lösung ›

Für Python ist eine Datei zunächst nur eine lange Folge von Zeichen. Von Spalten und Zahlen weiß es nichts, das ist Ihre Aufgabe.

`int(6.9)` schneidet ab, `int("6.9")` scheitert dagegen mit einem `ValueError`, weil in einem Text für `int` kein Punkt stehen darf. Runden können Sie mit `round(6.9)`.

Mit Text arbeiten

Texte kann man zusammensetzen, nach ihrer Länge fragen und Teile herausschneiden:

```
1  station = "Fichtelberg"
2  print("Station " + station)
3  print(len(station))
4
5  datum = "18900801"
6  jahr = datum[0:4]
7  monat = datum[4:6]
8  print(jahr, monat)
```

```
Another process is running, wait until finished
```

`datum[0:4]` bedeutet: die Zeichen ab Position 0 bis **vor** Position 4. Python beginnt beim Zählen immer bei **0**.

Stellen Sie sich die Positionen als Striche zwischen den Zeichen vor: `[0:4]` schneidet von Strich 0 bis Strich 4. So lässt sich die Länge direkt ablesen, 4 minus 0 sind vier Zeichen. Eine negative Position zählt von hinten, `station[-1]` ist das letzte Zeichen.

Vergleiche

Vergleiche liefern einen Wahrheitswert — `True` oder `False`:

Operator	Bedeutung	Beispiel	Ergebnis
`<`	kleiner	`-14.1 < 0`	`True`
`>`	größer	`-14.1 > 0`	`False`
`<=`	kleiner oder gleich	`0 <= 0`	`True`
`==`	gleich	`-999 == -999`	`True`
`!=`	ungleich	`5.2 != -999`	`True`

```
1  minimum = -3.5
2  frosttag = minimum < 0
3  print("Frosttag?", frosttag)
```

```
Frosttag? True
```

Achtung: `=` weist zu, `==` vergleicht. Die Verwechslung ist einer der häufigsten Anfängerfehler.

Mit diesen Wahrheitswerten trifft ein Programm **Entscheidungen** — darum geht es in der nächsten Vorlesung.

Vorsicht bei zwei Vergleichen: `"10" < "9"` ist `True`, weil Text Zeichen für Zeichen verglichen wird. Wandeln Sie deshalb vor dem Vergleichen um. Und `0.1 + 0.2 == 0.3` ist `False`, weil Kommazahlen im Rechner nicht exakt gespeichert werden. Kommazahlen sollte man deshalb nicht mit `==` auf exakte Gleichheit prüfen.

Kommentare

Alles hinter einem `#` ignoriert der Interpreter. Kommentare sind für Menschen:

```
1  fehlwert = -999       # So kennzeichnet der DWD "nicht gemessen"
2  minimum = -999
3
4  # Ein Fehlwert ist kein Frost, auch wenn -999 < 0 ist!
5  print(minimum < 0)
```

```
True
```

> Gute Kommentare erklären, **warum** etwas so ist — nicht **was** dasteht. `zaehler = zaehler + 1 # erhöhe zaehler um 1` hilft niemandem.

Das Programm zeigt das Problem, löst es aber noch nicht: `-999` ist kleiner als 0 und zählt damit als Frost. Wie man das verhindert, lernen Sie nächste Woche. In Visual Studio Code macht `Strg+/`, auf deutscher Tastatur oft `Strg+#`, aus markierten Zeilen Kommentare. So schaltet man Zeilen vorübergehend ab.

Fehlermeldungen lesen

Fehler sind normal — auch nach Jahren der Programmiererfahrung. Entscheidend ist, die Meldung zu **lesen**, statt sie wegzuklicken.

```
1  stationshoehe = 1213
2  print("Die Station liegt auf", stationshoe, "m Höhe")
```

```
Traceback (most recent call last):
  File "/lib/python313.zip/_pyodide/_base.py", line 597, in
eval_code_async
    await CodeRunner(
    ...<9 lines>...
    .run_async(globals, locals)
  File "/lib/python313.zip/_pyodide/_base.py", line 411, in run_async
    coroutine = eval(self.code, globals, locals)
  File "<exec>", line 2, in <module>
NameError: name 'stationshoe' is not defined. Did you mean:
'stationshoehe'?
```

Eine Python-Fehlermeldung lesen Sie **von unten nach oben**:

```
Traceback (most recent call last):
  File "<exec>", line 2, in <module>          ← wo: Zeile 2
NameError: name 'stationshoe' is not defined  ← was: diesen Namen gibt es
```

Die häufigsten Fehler am Anfang:

Fehler	Bedeutung	typische Ursache
`SyntaxError`	Der Text ist kein gültiges Python	Klammer oder Anführungszeichen vergessen
`NameError`	Diesen Namen kennt Python nicht	Tippfehler, Variable noch nicht angelegt
`TypeError`	Diese Operation passt nicht zu diesem Typ	`"-14.1" + 1`
`ValueError`	Der Typ stimmt, der Wert aber nicht	`float("6,26")`

Und dann? Lesen, Zeile anschauen, Hypothese bilden, ändern, erneut ausführen. Hilft das nicht: die letzte Zeile der Meldung in eine Suchmaschine kopieren. Sie sind garantiert nicht die erste Person mit diesem Fehler.

Traceback heißt Rückverfolgung. In Visual Studio Code steht statt `<exec>` der Pfad Ihrer Datei. Auch ein KI-Assistent kann eine Meldung erklären. Prüfen Sie seinen Vorschlag aber wie jedes andere Ergebnis: Können Sie sagen, warum er das Problem löst?

Typische Fehlvorstellung: Der Fehler steht in der gemeldeten Zeile

Aus Ihren Antworten in Vorlesung 00: *„… praktisch leider oft mini Schreibfehler, die niemand findet“*

Typische Fehlvorstellung: *„Der Fehler steht in der Zeile, die Python meldet.“*

```
1  minimun = -3.5
2  maximum = 0.7
```

```
3 spanne = maximum - minimum
4 print("Spanne:", spanne)
```

```
Spanne: 999.7
```

Legen Sie sich fest, **bevor** Sie das Programm ausführen: In welcher Zeile müssen Sie etwas ändern?

- ◯ in Zeile 3, denn dort meldet Python den Fehler
- ◯ in Zeile 1
- ◯ in Zeile 4

Bei einem `NameError` lohnt immer der Blick nach oben: Wo wurde der Name angelegt, und ist er dort genauso geschrieben? Ähnlich bei Klammern: Fehlt eine schließende Klammer, meldet Python `'(' was never closed` und zeigt auf die öffnende. Die Suche beginnt also an der gemeldeten Stelle, endet aber oft weiter oben.

Üben: Fehler finden

⏱ **5 Minuten, zu zweit.** Führen Sie jedes Programm aus, lesen Sie die letzte Zeile der Meldung und reparieren Sie das Programm direkt im Editor. Ziel ist jeweils die angegebene Ausgabe.

1. Soll `Fichtelberg` ausgeben:

```
1 station = "Fichtelberg
2 print(station)
```

```
Traceback (most recent call last):
  File "/lib/python313.zip/_pyodide/_base.py", line 597, in
eval_code_async
    await CodeRunner(
          ~~~~~~~~~~^
        source,
        ^^^^^^^
    ...<5 lines>...
        optimize=optimize,
        ^^^^^^^^^^^^^^^^^^
    )
    ^
  File "/lib/python313.zip/_pyodide/_base.py", line 285, in __init__
    self.ast = next(self._gen)
               ~~~~^^^^^^^^^^^
  File "/lib/python313.zip/_pyodide/_base.py", line 149, in
_parse_and_compile_gen
    mod = compile(source, filename, mode, flags | ast.PyCF_ONLY_AST)
  File "<exec>", line 1
    station = "Fichtelberg
              ^
SyntaxError: unterminated string literal (detected at line 1)
```

2. Soll `Messbeginn: 1890` ausgeben:

```
1  Print("Messbeginn:", 1890)
```

```
Traceback (most recent call last):
  File "/lib/python313.zip/_pyodide/_base.py", line 597, in
eval_code_async
    await CodeRunner(
    ...<9 lines>...
    .run_async(globals, locals)
  File "/lib/python313.zip/_pyodide/_base.py", line 411, in run_async
    coroutine = eval(self.code, globals, locals)
  File "<exec>", line 1, in <module>
NameError: name 'Print' is not defined. Did you mean: 'print'?
```

3. Soll `Höhe: 1213 m` ausgeben:

```
1  hoehe = 1213
2  print("Höhe: " + hoehe + " m")
```

```
Traceback (most recent call last):
  File "/lib/python313.zip/_pyodide/_base.py", line 597, in
eval_code_async
    await CodeRunner(
    ...<9 lines>...
    .run_async(globals, locals)
  File "/lib/python313.zip/_pyodide/_base.py", line 411, in run_async
    coroutine = eval(self.code, globals, locals)
  File "<exec>", line 2, in <module>
TypeError: can only concatenate str (not "int") to str
```

4. Soll `27.9` ausgeben:

```
1  maximum = float("26,9")
2  print(maximum + 1)
```

```
Traceback (most recent call last):
  File "/lib/python313.zip/_pyodide/_base.py", line 597, in
eval_code_async
    await CodeRunner(
    ...<9 lines>...
    .run_async(globals, locals)
  File "/lib/python313.zip/_pyodide/_base.py", line 411, in run_async
    coroutine = eval(self.code, globals, locals)
  File "<exec>", line 1, in <module>
ValueError: could not convert string to float: '26,9'
```

5. Soll `41.0` ausgeben. Hier hilft Ihnen keine Meldung:

```
1  spanne = 26,9 - -14.1
2  print(spanne)
```

```
(26, 23.1)
```

Lösung ›

Die fünf Programme enthalten absichtlich je einen Fehler, je einen der vier Typen aus der Tabelle und einen ohne Meldung. Gehen Sie immer gleich vor: letzte Zeile lesen, Zeilennummer anschauen, eine Vermutung bilden, ändern, erneut ausführen. Das letzte Beispiel zeigt, warum man die Ausgabe auch dann prüfen muss, wenn keine Meldung kommt.

Die Antwort auf die Leitfrage

So steht der 1. Januar 2024 in `fichtelberg_2024.csv`, die Spalten sind Datum, Tagesmittel, Minimum und Maximum:

```
#         012345678901234567890123456
zeile = "2024-01-01;-1,2;-1,9;-0,7"

minimum_str = zeile[16:20]    # von 16 bis 19, also "-1,9"
print(minimum_str, type(minimum_str))

minimum = float(minimum_str.replace(",", "."))
print(minimum, minimum < 0)
```

```
Another process is running, wait until finished
```

Für den Rechner ist `-1,9` zunächst nur Text. Erst wenn wir das richtige Stück **herausschneiden**, das Komma **ersetzen** und **umwandeln**, wird daraus eine Zahl, mit der man rechnen und vergleichen kann. In diesen Zeilen steckt die ganze heutige Vorlesung.

Warum Position 16? Das Datum belegt zehn Zeichen, dann folgen ein Semikolon, vier Zeichen Tagesmittel und noch ein Semikolon. Sehen Sie die Parallele zum Anfang der Vorlesung? Abspalten, umwandeln, vergleichen: Das waren dieselben drei Schritte wie bei den Uhrzeiten in der Tabellenkalkulation. Nur schreiben Sie sie jetzt selbst auf.

Typische Fehlvorstellung: Was für eine Zeile stimmt, stimmt für alle

Typische Fehlvorstellung: *„Läuft das Programm für eine Zeile richtig, läuft es für jede richtig.“*

Dasselbe Programm, sechs Tage später:

```
#         012345678901234567890123456
zeile = "2024-01-07;-8,9;-11,9;-4,2"

minimum_str = zeile[16:20]
minimum = float(minimum_str.replace(",", "."))
print(minimum)
```

```
Another process is running, wait until finished
```

Legen Sie sich fest, **bevor** Sie das Programm ausführen: Was wird ausgegeben?

- ◯ `-11.9`
- ◯ `-11.0`
- ◯ eine Fehlermeldung

Das ist derselbe Fehlertyp wie das Komma in `print(6,26 + 1)`: keine Meldung, aber ein falsches Ergebnis. Deshalb reicht es nicht, ein Programm an einer einzigen Zeile zu prüfen. Die saubere Lösung lernen Sie in Vorlesung 06 kennen: `split(";")` zerlegt eine Zeile an den Semikolons, egal wie lang die Zahlen sind.

Kontrollfragen

Was gibt dieses Programm aus?

```
x = 4
y = x + 3
x = y * 2
y = x - y
print(x, y)
```

Finden Sie alle Fehler.

```
Tagesmittel = 6.26
spanne = 26,9 - -14.1
print("Mittel:" tagesmittel)
```

- ☐ Zeile 2: `26,9` muss `26.9` heißen
- ☐ Zeile 3: Zwischen `"Mittel:"` und dem Namen fehlt ein Komma
- ☐ Zeile 3: `tagesmittel` ist klein geschrieben, angelegt wurde `Tagesmittel`
- ☐ Zeile 2: `- -14.1` ist nicht erlaubt

Weitere Kontrollfragen zum Nacharbeiten

Welchen Typ haben die folgenden Werte?

int	float	str	bo	
◯	◯	◯	◯	`1890`
◯	◯	◯	◯	`-0.5`
◯	◯	◯	◯	`"-0.5"`
◯	◯	◯	◯	`-0.5 < 0`
◯	◯	◯	◯	`7 / 7`
◯	◯	◯	◯	`18900801 // 10000`

Welche Fehlermeldung erzeugt `print("Frosttage: " + 127)`?

- ◯ `SyntaxError`
- ◯ `NameError`
- ◯ `TypeError`
- ◯ `ValueError`

Welche Zeilen sind gültige Zuweisungen?

- ☐ `hoehe = 1213`
- ☐ `1213 = hoehe`
- ☐ `hoehe + 1 = neue_hoehe`
- ☐ `neue_hoehe = hoehe + 1`
- ☐ `hoehe = hoehe + 1`

Ihre Begriffe für die Kurslandkarte

Die Kurslandkarte wächst mit dem Semester. Wir schlagen die Begriffe vor, Sie ergänzen, was fehlt: Aus Ihren Datenwünschen in Vorlesung 00 sind so schon die ersten Anwendungen geworden. Nehmen Sie sich dafür jetzt fünf Minuten.

Welches Wort von heute müssten Sie jemandem aus Ihrem Semester erklären können?

Texteingabe …

Welche Daten aus Ihrem Fach würden Sie gern so auswerten?

Texteingabe …

Nächste Woche

Ab der nächsten Vorlesung arbeiten wir zusätzlich mit Python auf Ihrem eigenen Rechner.

Zur Vorbereitung

- [ ] Falls in der ersten Übung noch nicht geschehen: Installieren Sie Python (mindestens Version 3.12) von python.org und Visual Studio Code mit der Erweiterung *Python*. Die Anleitung steht auf Blatt 1, Teil B.
- [ ] Schreiben Sie eine Datei `hallo.py` mit einer `print`-Anweisung und führen Sie sie aus.
- [ ] Probieren Sie: Wie rechnet man in Python eine Temperatur von °C in °F um? ($F = C \cdot 1{,}8 + 32$ F = C · 1,8 + 32)

> Wenn die Installation nicht klappt: Bringen Sie Ihren Laptop in die Übung mit. Dort helfen wir.

Zur Übung

Blatt 2 (Bohrkerne vermessen) steht nach der Vorlesung in OPAL. Versuchen Sie sich **vor** der Übung an den Aufgaben, gern auch zu zweit. Besprochen wird es in der Übung ab dem 09.11.